LACHOO MEMORIAL COLLEGE

OF SCIENCE AND TECHNOLOGY

PROJECT REPORT ON

Solar System Simulation

COURSE NAME: - Bachelor of Computer Applications (BCA)

COURSE CODE: - CCBCA421

SEMESTER: - IVth

GROUP MEMBERS: - Manvendra Singh Chouhan - 243501117
Lakshay Gangwani - 243501104
Mahesh Choudhary - 243501113

FACULTY GUIDE: - Mr. Aman Mathur

ABSTRACT

The Solar System Simulation using Python Turtle is a graphical project that demonstrates the motion of planets around the sun in a simplified and visually engaging manner. The primary aim of this project is to provide a basic understanding of planetary motion, animation, and coordinate-based movement using Python programming.

This project simulates the solar system where multiple planets revolve around a central sun at different speeds and distances. Each planet follows a circular orbit, and additional features such as a moon orbiting the Earth and Saturn's ring enhance the visual representation. The simulation uses mathematical functions such as sine and cosine to calculate circular motion, making it both educational and interactive.

The project is developed using Python and its built-in turtle graphics library, along with the math and random libraries. The turtle module is used to draw shapes and animate movement, while the math module helps in calculating orbital paths. The random module is used to generate background stars for a realistic space environment.

The final outcome of this project is an animated solar system model that runs continuously and allows the user to pause and resume the animation using keyboard input. This project successfully demonstrates the integration of programming logic, mathematics, and graphical visualization.

INTRODUCTION

- **Background : -**

  The solar system consists of the Sun and all celestial bodies orbiting around it, including planets, moons, asteroids, and comets. Understanding planetary motion is an important concept in both physics and astronomy. However, visualizing such motion can be challenging without graphical representation.

- **Motivation : -**

  The motivation behind this project is to create a simple and interactive simulation that visually demonstrates how planets revolve around the sun. It also helps students understand animation, coordinate systems, and mathematical concepts in programming.

- **Problem Statement : -**

  To design and implement a graphical simulation of the solar system using Python that shows planets moving in circular orbits with different speeds and distances from the sun.

- **Objectives : -**

  - To simulate planetary motion using Python Turtle
  - To implement circular motion using trigonometric functions
  - To create an animated graphical interface
  - To include additional features like moon orbit and labels
  - To improve understanding of animation logic

- **User Requirements : -**

  - Basic system capable of running Python
  - Python installed with turtle module
  - Keyboard interaction (spacebar for pause/play)

SYSTEM REQUIREMNTS

- **Hardware Requirements : -**

  Processor: Intel i3 or higher
  RAM: Minimum 4 GB
  Storage: 500 MB free space
  Display: Standard monitor

- **Software Requirements : -**

  Operating System: Windows / Linux / Mac
  Programming Language: Python 3.x
  IDE: VS Code / PyCharm / IDLE

- **Libraries Used : -**

  turtle → Used for graphics and animation
  math → Used for trigonometric calculations (sin, cos)
  random → Used for generating stars

METHODOLOGY / PROJECT DESIGN

- **Project Flow : -**

  1. Initialize the turtle screen with a black background
  2. Draw the sun at the center
  3. Generate random stars in the background
  4. Draw circular orbits for each planet
  5. Create planets with different sizes and colors
  6. Apply circular motion using sine and cosine functions
  7. Add moon orbiting Earth
  8. Add labels for each planet
  9. Animate continuously using a loop
  10. Allow pause/resume using keyboard input

- **Working Principle : -**

  The project uses circular motion formulas:

  $$x = r \times \cos(angle)$$

  $$y = r \times \sin(angle)$$

  Each planet has:

  1. Different radius (distance from sun)

  2. Different angular speed

  This creates realistic motion where planets move at different ` speeds.

- **Modules Used : -**

  1. Input Module - Handles user interaction (keyboard input like spacebar to pause/resume animation).

  2. Processing Module - Contains the main logic including mathematical calculations for motion.

3. **Output Module -** Displays planets, orbits, and animations using turtle graphics.

4. **Animation Module -** Controls continuous movement using loops and angle updates.

- **Libraries Explanation :**

turtle: Used for drawing shapes and animation

math: Provides sin() and cos() for circular motion

random: Used to generate random star positions

RESULT AND ANALYSIS

The project successfully simulates a basic solar system with multiple planets orbiting around the sun. Each planet moves at a different speed, which creates a realistic animation effect.

- **Key Features Achieved : -**
  - Smooth animation of planets
  - Different orbital speeds
  - Moon orbit around Earth
  - Saturn ring visualization
  - Planet labels
  - Background star

- **Observations : -**
  - Inner planets move faster than outer planets.
  - Circular motion appears smooth due to continuous angle updates.
  - The simulation is visually appealing and interactive.

- **Limitations : -**
  - Not scaled to real astronomical distances
  - No elliptical orbits
  - No gravitational physics
  - Limited to 2D representation

CONCLUSION AND FUTURE SCOPE

- **Conclusion : -**

The Solar System Simulation project successfully demonstrates the concept of planetary motion using Python Turtle graphics. It combines programming, mathematics, and visualization to create an interactive learning tool.

The project helped in understanding:

- Animation logic
- Trigonometric motion
- Graphics programming

- **Future Scope : -**

- Add realistic scaling of planets
- Implement elliptical orbits
- Add 3D visualization
- Include user controls (zoom, speed control)
- Use advanced libraries like Pygame or OpenGL
- Add sound effects and GUI

REFERENCES

- **Python Official Documentation – https://docs.python.org**
- **Turtle Graphics Documentation**
- **W3Schools Python Tutorials**
- **GeeksforGeeks Python Graphics Articles**

APPENDIX

➢ Source Code : -

```
import turtle
import math
import random

# Screen setup
screen = turtle.Screen()
screen.bgcolor("black")
screen.title("Solar System Simulation")
screen.tracer(0)

# Pause control
running = True
def toggle():
    global running
    running = not running
screen.listen()
screen.onkey(toggle, "space")

# Sun
sun = turtle.Turtle()
sun.shape("circle")
sun.color("orange")
sun.shapesize(2.5)
sun.penup()
```

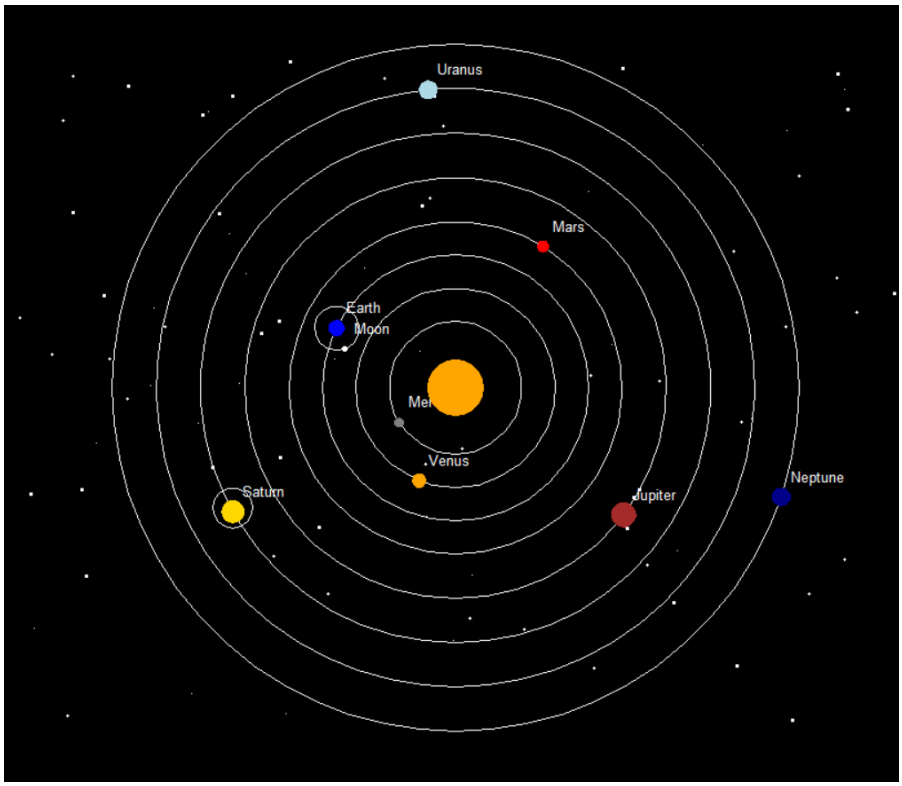
```
# Stars background
stars = turtle.Turtle()
stars.hideturtle()
stars.speed(0)
stars.penup()

for _ in range(80):
    x = random.randint(-400, 400)
    y = random.randint(-300, 300)
    stars.goto(x, y)
    stars.dot(random.randint(2, 4), "white")

# Orbit function
def draw_orbit(radius):
    orbit = turtle.Turtle()
    orbit.hideturtle()
    orbit.color("white")
    orbit.penup()
    orbit.goto(0, -radius)
    orbit.pendown()
    orbit.circle(radius)

# Planet function
def create_planet(color, size, name):
    p = turtle.Turtle()
    p.shape("circle")
    p.color(color)
    p.shapesize(size)
    p.penup()
    label = turtle.Turtle()
    label.hideturtle()
```

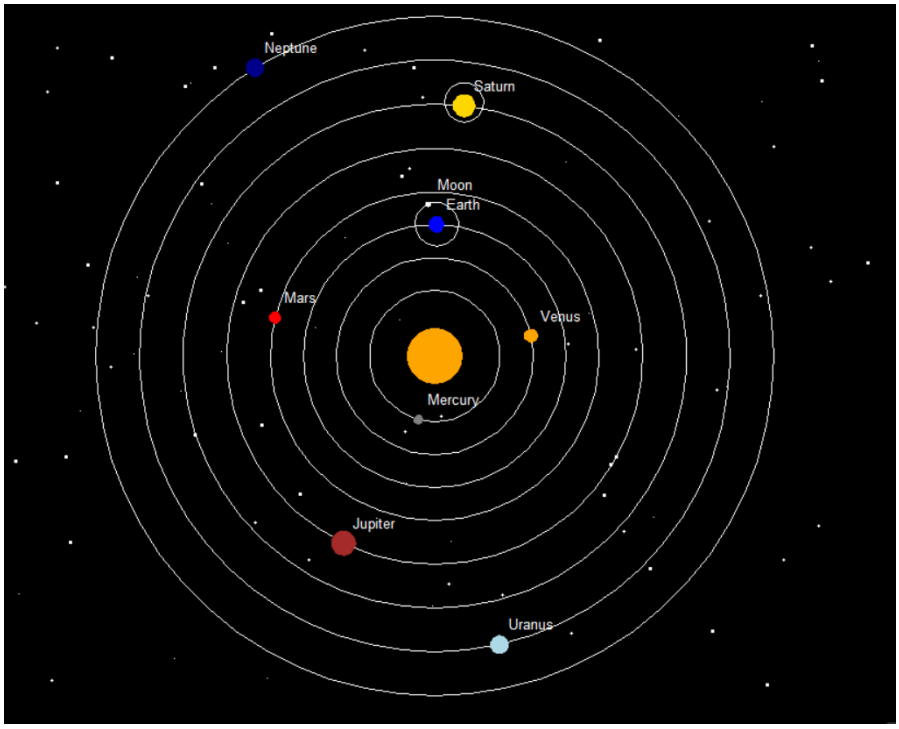
```
    label.color("white")
    label.penup()

    return p, label

# Creating planets + labels
mercury, m_label = create_planet("gray", 0.4, "Mercury")
venus, v_label   = create_planet("orange", 0.6, "Venus")
earth, e_label   = create_planet("blue", 0.7, "Earth")
mars, ma_label   = create_planet("red", 0.5, "Mars")
jupiter, j_label = create_planet("brown", 1.1, "Jupiter")
saturn, s_label  = create_planet("gold", 1.0, "Saturn")
uranus, u_label  = create_planet("lightblue", 0.8, "Uranus")
neptune, n_label = create_planet("darkblue", 0.8, "Neptune")

# Orbit radii
r = [60, 90, 120, 150, 190, 230, 270, 310]

# Drawing orbits
for i in r:
    draw_orbit(i)

#Moon
moon = turtle.Turtle()
moon.shape("circle")
moon.color("white")
moon.shapesize(0.2)
moon.penup()
```

```
# Saturn rings
ring = turtle.Turtle()
ring.hideturtle()
ring.color("white")
ring.penup()

moon_label = turtle.Turtle()
moon_label.hideturtle()
moon_label.color("white")
moon_label.penup()

moon_orbit_radius = turtle.Turtle()
moon_orbit_radius.hideturtle()
moon_orbit_radius.color("white")
moon_orbit_radius.penup()
angle = 0
while True:
    if running:
        mercury.goto(r[0]*math.cos(angle*1.2), r[0]*math.sin(angle*1.2))
        venus.goto(r[1]*math.cos(angle*0.9),   r[1]*math.sin(angle*0.9))
        earth_x = r[2]*math.cos(angle*0.7)
        earth_y = r[2]*math.sin(angle*0.7)
        earth.goto(earth_x, earth_y)
        moon.goto(earth_x + 20*math.cos(angle*2.5), earth_y + 20*math.sin(angle*2.5))
        moon_orbit_radius.clear()
        moon_orbit_radius.goto(earth_x, earth_y-20)
        moon_orbit_radius.pendown()
        moon_orbit_radius.circle(20)
        moon_orbit_radius.penup()
        mars.goto(r[3]*math.cos(angle*0.5),    r[3]*math.sin(angle*0.5))
        jupiter.goto(r[4]*math.cos(angle*0.3), r[4]* math.sin(angle*0.3))
```

```
    saturn.goto(r[5]*math.cos(angle*0.25), r[5]*math.sin(angle*0.25))
    uranus.goto(r[6]*math.cos(angle*0.2),  r[6]*math.sin(angle*0.2))
    neptune.goto(r[7]*math.cos(angle*0.15),r[7]*math.sin(angle*0.15))

    # Labels
    def label_planet(label, planet, name):
        label.goto(planet.xcor()+10, planet.ycor()+10)
        label.clear()
        label.write(name, font=("Arial", 10, "normal"))

    label_planet(m_label, mercury, "Mercury")
    label_planet(v_label, venus, "Venus")
    label_planet(e_label, earth, "Earth")
    label_planet(moon_label, moon, "Moon")
    label_planet(ma_label, mars, "Mars")
    label_planet(j_label, jupiter, "Jupiter")
    label_planet(s_label, saturn, "Saturn")
    label_planet(u_label, uranus, "Uranus")
    label_planet(n_label, neptune, "Neptune")

    # Saturn ring
    ring.clear()
    ring.goto(saturn.xcor(), saturn.ycor()-15)
    ring.pendown()
    ring.circle(18)
    ring.penup()
    angle += 0.01
screen.update()
```

➢ Screenshots : -





- **GitHub Link : -**

https://github.com/mansa022005/College-Projects/tree/main/Computer%20Graphics%20IVth%20Sem%20Mini%20Project